1. วัตถุประสงค์และความคืบหน้า

RescueLink เป็นแอป Flutter สำหรับแจ้งเหตุ SOS และสื่อสารระหว่างอุปกรณ์เมื่ออินเทอร์เน็ตไม่พร้อม ใช้ SQLite เก็บข้อมูลในเครื่อง และ Supabase สำหรับสมาชิก ข้อมูลเหตุ SOS และไฟล์ที่ผู้ใช้เลือกอัปโหลด

งาน Week 3: สาธิตการเชื่อมต่อฐานข้อมูลและ CRUD สมาชิกผ่าน Chrome ได้แก่ สมัครสมาชิก เข้าสู่ระบบ อ่านโปรไฟล์ แก้ไขชื่อ และลบบัญชี พร้อมรายงาน schema และส่วนประกอบของระบบ

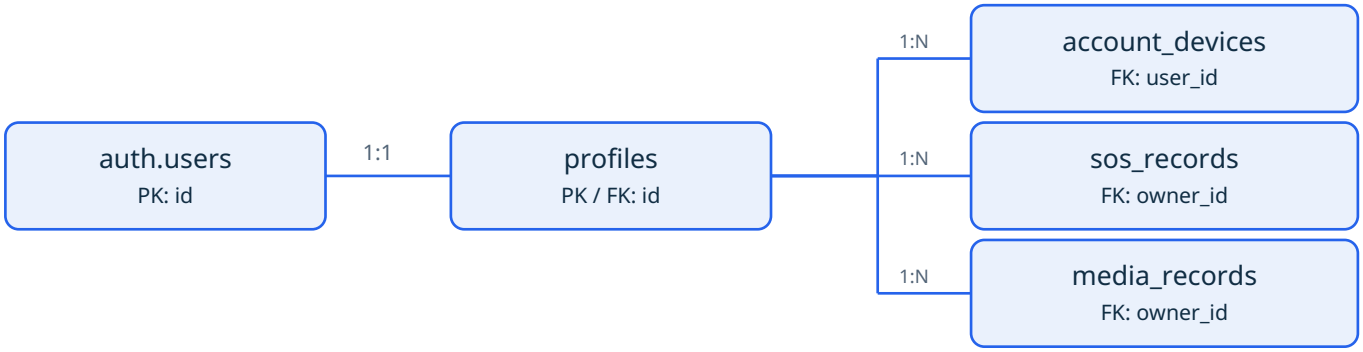
หัวข้อ	สิ่งที่พัฒนาแล้ว
Create / Auth	สมัครอีเมลและรหัสผ่าน พร้อมชื่อสมาชิก; หากเปิด email confirmation ต้องยืนยันอีเมลก่อน Login
Read	หน้า “ข้อมูลสมาชิก” อ่าน id และ display_name จาก profiles บน Supabase; อีเมลมาจาก session
Update	แก้ชื่อ 1-100 ตัวอักษร และบันทึกเฉพาะโปรไฟล์ของบัญชีปัจจุบัน
Delete	Edge Function delete-account ตรวจ session และรหัสผ่าน ลบไฟล์ Cloud ของเจ้าของ เพิกถอน session และลบบัญชี Auth
Local / Sync	ข้อมูล SOS และคิวอยู่ใน SQLite; มีการส่งซ้ำ ตรวจเวอร์ชัน และจัดการ conflict

สถานะการยืนยัน: Flutter ทั้งโปรเจกต์ผ่าน 144 tests และข้าม 4 live tests ที่ต้องเปิดใช้งานเอง; บริการลบบัญชีผ่าน 8 tests แบบจำลอง Analyzer ไม่มีปัญหาและ build เว็บผ่าน ตรวจ endpoint จริงพบคำขอไม่มี session ได้ 401 และ CORS preflight ได้ 204

การสมัคร ยืนยันอีเมล และลบบัญชีจริงแบบครบวงจรยังต้องซ้อมด้วยบัญชีทดสอบของผู้ใช้ก่อนบันทึกวิดีโอ ผลทดสอบจำลองไม่ได้เป็นหลักฐานว่าขั้นตอนนี้ผ่านบน Cloud จริงทั้งหมด

2. Schema และความสัมพันธ์

Schema สมาชิกบน Supabase (PostgreSQL) ใช้ UUID ของ auth.users เป็นตัวเชื่อมหลัก



ตารางต้นทาง	ความสัมพันธ์	ตารางปลายทาง / Foreign key
auth.users	1 : 1	profiles.id references auth.users.id
profiles	1 : N	account_devices.user_id references profiles.id
profiles	1 : N	sos_records.owner_id references profiles.id
profiles	1 : N	media_records.owner_id references profiles.id
account_devices	1 : N	owned_records.(owner_id, installation_id) references account_devices.(user_id, installation_id)

Primary key: profiles ใช้ id; account_devices ใช้ (user_id, installation_id); sos_records และ media_records ใช้ (owner_id, id)

Foreign key หลักกำหนด ON DELETE CASCADE เมื่อบัญชีใน auth.users ถูกลบ โปรไฟล์และข้อมูลลูกที่เกี่ยวข้องจึงถูกลบตาม ส่วน Storage ต้องลบไฟล์ผ่าน Storage API ก่อนลบบัญชี

ฐานข้อมูลในเครื่อง: accounts.db เก็บรหัสการติดตั้งและบัญชีที่เป็นเจ้าของไฟล์ฐานข้อมูล แต่ละบัญชีมีไฟล์ SQLite แยกกัน ภายในมี settings, peers, messages, sos_records, sos_queue และข้อมูลสื่อ/คิว relay

ข้อมูล session เก็บผ่าน flutter_secure_storage ไม่เก็บรหัสผ่านใน SQLite ไม่ฝังกุญแจ service_role หรือ secret key ในแอป

ตาราง owned_records เป็นโครงสร้างเตรียมไว้ การมีตารางนี้ไม่หมายถึงข้อความแชตถูกซิงก์แล้ว ข้อความแชตปัจจุบันยังเก็บและรับส่งระหว่างเครื่อง

3. Data dictionary

ตาราง	คอลัมน์ / ชนิด	ความหมาย
profiles	id: uuid (PK, FK) display_name: text created_at: timestamptz	โปรไฟล์สมาชิก ชื่อยาวไม่เกิน 100 ตัวอักษร; trigger สร้างแถวหลังสมัคร
account_devices	user_id: uuid installation_id: uuid radio_identity: uuid linked_at, last_seen_at: timestamptz	บัญชีที่ผูกกับการติดตั้งแอป และรหัสสำหรับสื่อสาร Nearby; ไม่ใช่รหัสฮาร์ดแวร์
sos_records (Cloud)	owner_id, id: uuid payload: jsonb version: bigint deleted: boolean updated_at: timestamptz	เหตุ SOS ของเจ้าของ พร้อมเวอร์ชันและเครื่องหมายลบ; payload เก็บชื่อ ประเภท จำนวนคน รายละเอียด และพิกัดถ้ามี
media_records	owner_id, id, message_id: uuid file_name, mime_type, checksum: text file_size: bigint	ข้อมูลไฟล์ที่อัปโหลดสำเร็จและตรวจ checksum แล้ว ไฟล์จริงอยู่ใน bucket chat-media
sos_queue (SQLite)	operation_id, record_id: TEXT base_version: INTEGER payload: TEXT deleted, attempts, blocked: INTEGER error: TEXT	คิวคำสั่งรอส่งบนเครื่อง ใช้ operation_id กันคำสั่งซ้ำ และ base_version ตรวจ conflict
messages (SQLite)	id, senderId, receiverId, text, timestamp, type, status	ข้อความและสถานะการส่งระหว่างอุปกรณ์ เก็บในฐานข้อมูลเครื่องของบัญชี

4. การทำ CRUD สมาชิก

Create: ฟอร์มตรวจชื่อ อีเมล รหัสผ่าน และการยืนยันรหัสผ่าน -> Supabase Auth signUp -> auth.users -> trigger สร้าง profiles -> ผู้ใช้ยืนยันอีเมลตามการตั้งค่าระบบ

Login: signInWithPassword -> ได้ session / User UID -> เลือกฐานข้อมูล SQLite ของบัญชี -> upsert account_devices -> Chrome เปิดหน้าข้อมูลสมาชิก

Read: หน้า “ข้อมูลสมาชิก” ใช้ SELECT id, display_name FROM profiles โดยกรอง id เท่ากับบัญชีที Login พร้อมแสดงอีเมลและ UID ปุ่มโหลดใหม่อ่านค่าจาก Cloud อีกครั้ง

Update: ตรวจชื่อไม่ว่างและไม่เกิน 100 ตัวอักษร -> UPDATE profiles SET display_name โดยกรอง id ของตนเอง -> อ่านค่าทีฐานข้อมูลตอบกลับ -> แสดงข้อความบันทึกสำเร็จ

Delete: ผู้ใช้เลือก “ลบบัญชีถาวร” และกรอกรหัสผ่าน -> แอปเรียก Edge Function delete-account -> เซิร์ฟเวอร์ตรวจ JWT กับ Auth และตรวจรหัสผ่านอีกครั้ง -> ใช้เฉพาะ user.id ทีตรวจได้ -> ลบไฟล์ Cloud ของบัญชี -> เพิกถอน session -> Admin API ลบ auth.users -> FK ลบข้อมูลลูก -> แอปออกจากบัญชีและลบฐานข้อมูลของเจ้าของในเครือง

การป้องกันสิทธิ: ใช้ RLS profiles_owner โดย auth.uid() ต้องตรงกับ profiles.id ทั้ง USING และ WITH CHECK; Edge Function เปิด verify_jwt และตรวจตัวตนซ้ำก่อนงานทีใช้กุญแจผู้ดูแล

Logout เพียงจบ session และกลับ Guest ไม่ใช่ Delete ส่วนรหัสผ่านของฟอร์มยืนยันลบใช้ส่งเพือยืนยันกับ Auth และไม่บันทึกลง log หรือฐานข้อมูลของแอป

5. Sync, error handling และข้อจำกัด

SOS ทำงานตามเส้นทาง UI -> SQLite sos_records + sos_queue ภายใน transaction -> sync_sos บน Supabase -> Cloud ยืนยันผล -> ลบคำสั่งจากคิว -> ดึงข้อมูล Cloud กลับมา merge

กรณี	พฤติกรรม
ไม่มีอินเทอร์เน็ต	SOS ยังบันทึกในเครื่องได้ คิวคงอยู่และลองซิงก์ใหม่เมือเชือมต่อได้; โปรไฟล์สมาชิกต้องออนไลน์
ผลตอบกลับหาย / ส่งซ้ำ	ใช้ operation_id เดิมและ receipt ฝัง Cloud ป้องกันทำรายการซ้ำ
แก้ไขหลายเครือง	ตรวจ base_version; เมือ conflict ให้ผู้ใช้เลือกข้อมูลเครืองหรือ Cloud
ลบ SOS	เก็บ deleted = true บน Cloud ป้องกันข้อมูลเก่าจากเครืองออฟไลน์กลับมาสร้างรายการ
ชือว่าง / รหัสผิด	ฟอร์มไม่บันทึกชือว่าง; การยืนยันลบด้วยรหัสผิดไม่ลบบัญชี
โหลด / บันทึกไม่สำเร็จ	แสดง error และให้ลองใหม่ ไม่แสดงสำเร็จก่อนฐานข้อมูลตอบกลับ

การทดสอบ: Flutter regression ทังโปรเจกต์ผ่าน 144 tests ข้าม 4 live tests; Node 8 tests ของบริการลบบัญชีผ่าน ครอบคลุมการยกเลิก/รหัสผิด ตัวตนผิด การปลอมเป้าหมาย การลบเฉพาะไฟล์เจ้าของ และการลบฐานข้อมูลโดยไม่กระทบบัญชีอืน Supabase security advisor ไม่พบรายการแจ้งเตือน

ขอบเขตทียังไม่ได้ทำ: ข้อความแชตยังไม่มี Cloud Sync; Nearby ต้องทดสอบบน Android จริง; บัญชีสมาชิก CRUD บน Chrome ต้องซ้อมผ่านอีเมลจริงก่อนอัด ส่วนไฟล์สือเก่าทีเก็บในเครือง Androidไม่ได้ถูกลบเป็นไฟล์โดยฟังก์ชันลบฐานข้อมูลนี

6. ลำดับวิดีโอและหลักฐาน

ลำดับ	สิ่งที่แสดง / สิ่งที่พูด
1. แนะนำ	RescueLink ใช้ SQLite และ Supabase วันนี้สาธิต CRUD สมาชิกผ่าน Chrome
2. สมัคร	สร้างบัญชีทดสอบชื่อ Week3 Demo; แสดง Users และ profiles ของบัญชีใหม่ ยืนยันอีเมลก่อน Login ถ้าจำเป็น
3. Login / Read	ลองรหัสผิดหนึ่งครั้ง แล้ว Login ถูกต้อง; แสดงหน้าข้อมูลสมาชิก อีเมล UID และชื่อที่อ่านจากฐานข้อมูล
4. Update	แก้ชื่อเป็น Week3 Updated แล้วบันทึก; Refresh profiles ให้เห็น display_name เปลี่ยน โดย id เดิม
5. Schema	เปิด Schema / PDF อธิบาย auth.users -> profiles -> account_devices และข้อมูลเหตุ SOS
6. Delete	ลบบัญชีทดสอบผ่านแอปด้วยรหัสผ่านปัจจุบัน; แสดงว่ากลับหน้า Login และแถวบัญชี/โปรไฟล์หายจาก Supabase
7. ปิดท้าย	อธิบาย SOS local-first + คิว Sync และแยกแชต Nearby ที่ยังไม่ซิงก์ Cloud

ก่อนอัด: ใช้บัญชีทดสอบที่ยอมลบได้ ซ้อมครบหนึ่งรอบ เตรียม Supabase project RescueLink และตาราง profiles, account_devices เปิดไว้ ไม่โชว์รหัสผ่านหรือ token; ใส่รายชื่อสมาชิกกลุ่มในระบบส่งงานหรือหน้าปกเพิ่มเติมตามรูปแบบของวิชา

แหล่งอ้างอิง

Supabase: User Management
https://supabase.com/docs/guides/auth/managing-user-data
Securing Edge Functions
https://supabase.com/docs/guides/functions/auth
Auth Admin deleteUser
https://supabase.com/docs/reference/javascript/auth-admin-deleteuser